

Parsing

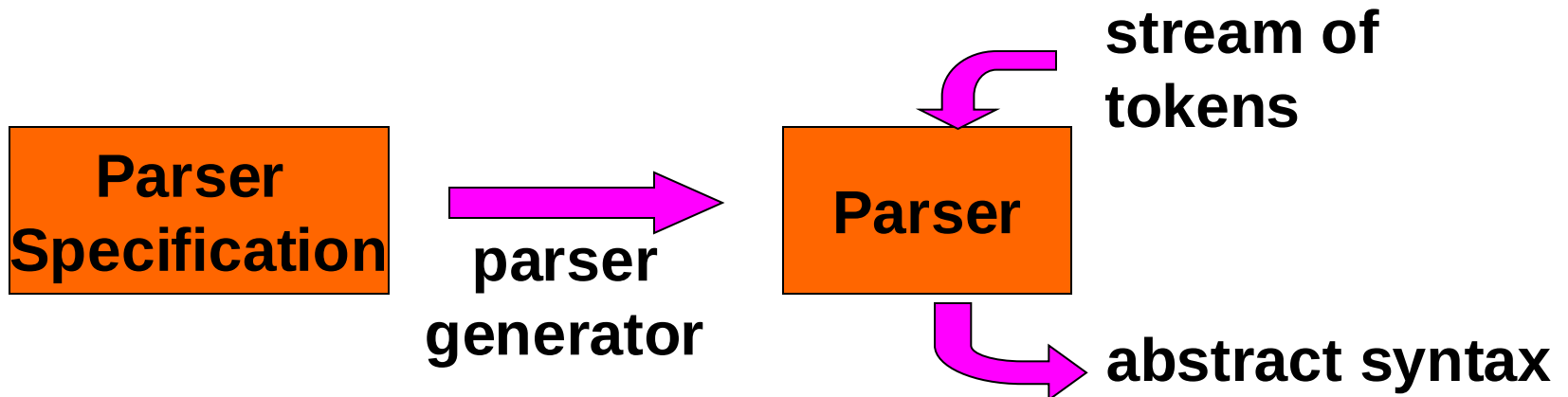
Chapter 3

Outline: How to build a parser?

- Specifying the syntax of a programming language with Context-Free Grammars (CFG)
- Build the parser based on the CFG:
 - Top-Down Parsing
 - Predictive Parsing (LL(k) Parsing)
 - Bottom-Up Parsing
 - LR Parsing
- More about parsing:
 - **Automatic parser generation**
 - **Error Recovery**

Parser Implementation

- Write a parser from scratch
- Can we automatically generate parsers?
 - LL(k) and LR(k) both have general table-driven parsing algorithms.
 - Different grammars lead to different parsing tables.
 - We have learned how to automatically construct parsing tables.
- Use a Parser Generator
 - general and robust
 - sometimes not quite as efficient as hand-written parsers
 - nevertheless, good for lazy compiler writers.



Yacc

- **Yacc**: Yet another compiler-compler
- **Input**: a specification file (usually with a suffix **.y**)
- **Output**: an output file consisting of C source code for the parser (usually with a suffix **tab.c**)
- A Yacc specification file has the basic format

{definitions}

%%

{rules}

%%

{auxiliary routines}

Yacc: An Example

exp $\rightarrow$ *exp addop term* | *term*

addop $\rightarrow$ + | -

term $\rightarrow$ *term mulop factor* | *factor*

mulop $\rightarrow$ *

factor $\rightarrow$ (*exp*) | number

Yacc: An Example

```
%{  
#include <stdio.h>  
#include <ctype.h>  
int yylex(void);  
int yyerror (char * s);  
%}  
%token NUMBER  
%%  
command: exp {printf("%d\n", $1);};  
exp: exp '+' term {$$ = $1 + $3;}  
   | exp '-' term {$$ = $1 - $3;}  
   | term {$$ = $1}  
;  
term: term '*' factor {$$ = $1 * $3;}  
     | factor {$$ = $1;}  
;  
factor: NUMBER {$$ = $1;}  
       | '(' exp ')' {$$ = $2;}  
;
```

```
%%  
int main() {  
    return yyparse();  
}  
  
int yylex(void){  
    int c;  
    // eliminate blanks  
    while ( (c=getchar()) == ' ');  
    if (isdigit(c)) {  
        ungetc(c, stdin);  
        scanf("%d", &yylval);  
        return (NUMBER);  
    }  
    if (c == '\n')  
        return 0; // stop the parse  
    return c;  
}  
  
int yyerror (char * s){  
    fprintf (stderr, "%s\n",s );  
    return 0;  
}
```

Yacc: Auxiliary Routines

- `yyparse` is declared to return an integer value, which is `0` if the parse succeeds, and `1` if it does not.
- The `yyparse` procedure calls a lexer procedure (`yylex`)
- Yacc expects the `end of input` to be signaled by a return of the null value `0` by `yylex`.
- `yylex` returns a token type or `0` and `yylval` stores the `semantic value` (pushed to the value stack later)
- `yyerror` prints an error message when an error is encountered during the parse.

```
%%  
int main() {  
    return yyparse();  
}  
  
int yylex(void){  
    int c;  
    // eliminate blanks  
    while ( (c=getchar()) == ' ');  
    if (isdigit(c)) {  
        ungetc(c, stdin);  
        scanf("%d", &yylval);  
        return (NUMBER);  
    }  
    if (c == '\n')  
        return 0; // stop the parse  
    return c;  
}  
  
int yyerror (char * s){  
    fprintf (stderr, "%s\n",s );  
    return 0;  
}
```

Yacc: Definitions

```
%{  
#include <stdio.h>  
#include <ctype.h>  
int yylex(void);  
int yyerror (char * s);  
%}  
%token NUMBER  
%%  
command: exp {printf("%d\n", $1);};  
exp: exp '+' term {$$ = $1 + $3;}  
   | exp '-' term {$$ = $1 - $3;}  
   | term {$$ = $1}  
;  
term: term '*' factor {$$ = $1 * $3;}  
     | factor {$$ = $1;}  
;  
factor: NUMBER {$$ = $1;}  
       | '(' exp ')' {$$ = $2;}  
;
```

Two ways of recognizing tokens:

- Any character inside single quotes in a grammar rule will be recognized as itself. e.g., '+'
- Symbolic tokens may be declared in a YACC %token declaration .
 - %token NUMBER
 - %start symbol (define the start symbol .)

Yacc: Rules

```
%{  
#include <stdio.h>  
#include <ctype.h>  
int yylex(void);  
int yyerror (char * s);  
%}  
%token NUMBER  
%%  
command: exp {printf("%d\n", $1);}  
exp: exp '+' term {$$ = $1 + $3;}  
  | exp '-' term {$$ = $1 - $3;}  
  | term {$$ = $1}  
;  
term: term '*' factor {$$ = $1 * $3;}  
  | factor {$$ = $1;}  
;  
factor: NUMBER {$$ = $1;}  
  | '(' exp ')' {$$ = $2;}  
;
```

Rules:

- Rule {Action Code}
 - similar to lex
- Action code will be executed after the parser performs a reduce action using the rule.
- Action code is placed at the end of each grammar rule choice. It is also possible to write **embedded actions** within a choice.

Yacc: Rules

```
%{  
#include <stdio.h>  
#include <ctype.h>  
int yylex(void);  
int yyerror (char * s);  
%}  
%token NUMBER  
%%  
command: exp {printf("%d\n", $1);};  
exp: exp '+' term {$$ = $1 + $3;}  
   | exp '-' term {$$ = $1 - $3;}  
   | term {$$ = $1}  
;  
term: term '*' factor {$$ = $1 * $3;}  
     | factor {$$ = $1;}  
;  
factor: NUMBER {$$ = $1;}  
       | '(' exp ')' {$$ = $2;}  
;
```

Pseudovariables: \$\$, \$1, \$3

- yylex may store the semantic value of a token in yylval
 - by default, yylval = 0

YYSTYPE yylval;

- When a grammar rule is recognized, each symbol in the rule possesses a value.
 - $\$i$: the i _th symbol in the RHS of the rule
 - $$$$: the value of the LHS of the rule
- These values are kept on the top of a value stack by Yacc.
- What will happen during reduce?

Yacc: Rules

```
%{  
#include <stdio.h>  
#include <ctype.h>  
int yylex(void);  
int yyerror (char * s);  
%}  
%token NUMBER  
%%  
command: exp {printf("%d\n", $1);};  
exp: exp '+' term {$$ = $1 + $3;}  
   | exp '-' term {$$ = $1 - $3;}  
   | term {$$ = $1}  
;  
term: term '*' factor {$$ = $1 * $3;}  
     | factor {$$ = $1;}  
;  
factor: NUMBER {$$ = $1;}  
       | '(' exp ')' {$$ = $2;}  
;
```

Example: 3 * 4

Action	Symbol Stack	Value Stack
shift	NUM	3 (from yylval)
reduce	factor	3
reduce	term	3
shift	term, '*'	3, 0 (default)
shift	term, '*', NUM	3, 0, 4
reduce	term, '*', factor	3, 0, 4
reduce	term	12
reduce	exp	12

Yacc: Rules

```
%{  
#include <stdio.h>  
#include <ctype.h>  
int yylex(void);  
int yyerror (char * s);  
%}  
%token NUMBER  
%%  
command: exp {printf("%d\n", $1);}  
exp: exp '+' term {$$ = $1 + $3;}  
  | exp '-' term {$$ = $1 - $3;}  
  | term {$$ = $1}  
;  
term: term '*' factor {$$ = $1 * $3;}  
  | factor {$$ = $1;}  
;  
factor: NUMBER {$$ = $1;}  
  | '(' exp ')' {$$ = $2;}  
;
```

Pseudovariables: \$\$, \$1, \$3

- yylex may store the semantic value of a token in yylval
 - by default, yylval = 0

YYSTYPE yylval;

- This data type is always defined in Yacc by the C preprocessor symbol YYSTYPE.
- By default:

```
#define YYSTYPE int
```

Yacc: YYSTYPE

$$\begin{aligned} \text{exp} &\rightarrow \text{exp addop term} \mid \text{term} \\ \text{op} &\rightarrow '+' \mid '-' \end{aligned}$$

- For the values of *exp* and *op*, what are their types?
- **Different value types for different grammar symbols**
- How to handle this?
 - Declare the union directly in the Yacc specification using the `%union` Yacc declaration:

```
%union { double val;  
char op;}
```

- Define a new data type in the definitions or in a separate include file, and **define YYSTYPE to this type**
 - The appropriate values must be constructed by hand in the associated action code.

```
#define YYSTYPE ASTNode
```

Yacc: %union & %type

```
%token NUMBER
%union { double val;
        char op;}
%type <val> exp term NUMBER
%type <op> op
%%
command : exp { printf("%f\n", $1); }
;
exp : exp op term {
    switch ($2){
        case '+': $$=$1+$3; break;
        case '-': $$=$1-$3; break;
    }
}
| term {$$ = $1;}
;
op : '+' {$$ = $1;}
| '-' {$$ = $1;}
;
```

- All **non-terminals** obtain their values by such user supplied actions
- **Tokens** may also be assigned values, this is done in the Lexer using `yylval`.
- Use **%union** to define value types
- Use **%type** to specify the value type of each symbol
 - **necessary**

$$$ = \$1 + \$3$

`(yyval.val)=(yyvsp[1-3].val) + (yyvsp[3-3].val)`

Yacc: Embedded Actions

- It is necessary to execute some code **prior to the complete recognition of a grammar rule** choice during parsing.

decl → *type var-list*

type → *int* | *float*

var-list → *var-list, id* | *id*

- How can we record the “type” of each “id”?

Yacc: Embedded Actions

```
decl: type {current_type=$1} var-list
```

```
;
```

```
type: INT {$$=INT_TYPE;}
```

```
| FLOAT {$$=FLOAT_TYPE;}
```

```
;
```

```
var_list: var_list ',' ID {setType(tokenString, current_type);}
```

```
| ID {setType(tokenString, current_type);}
```

```
;
```

```
list: item1 { do_item1($1); } item2 { do_item2($3); } item3
```

```
list: item1 _rule01 item2 _rule02 item3
```

```
_rule01: { do_item1($0); }
```

```
_rule02: { do_item2($0); }
```

Yacc: Conflict

- Yacc reports shift-reduce and reduce-reduce conflicts
- By default, Yacc resolves:
 - **shift-reduce conflicts** by **shifting**
 - **reduce-reduce conflicts** by **using the rule that appears earlier in the grammar**

```
state 17: shift/reduce conflict
          (shift ELSE, reduce 4)
stm : IF ID THEN stm .
stm : IF ID THEN stm . ELSE stm
ELSE  shift 19
.      reduce by rule 4
```

- Most shift-reduce conflicts, and all reduce-reduce conflicts, are serious problems and should be eliminated by rewriting the grammar.

Yacc: Precedence Directives

exp $\rightarrow$ *exp addop term* | *term*

addop $\rightarrow$ + | -

term $\rightarrow$ *term mulop factor* | *factor*

mulop $\rightarrow$ * | /

factor $\rightarrow$ (*exp*) | number

E $\rightarrow$ id

E $\rightarrow$ num

E $\rightarrow$ *E* * *E*

E $\rightarrow$ *E* / *E*

E $\rightarrow$ *E* + *E*

E $\rightarrow$ *E* - *E*

E $\rightarrow$ (*E*)

- The left grammar is not easy to understand and not very natural.
- Is there a way that we can write **natural grammar** and also respect **operator precedence and associativity**?
- Yacc has **precedence directives** (disambiguating rules) built into it.

Yacc: Precedence Directives

- The grammar in the left is natural but highly ambiguous. There are many shift-reduce conflicts in its LR(1) parsing table.

$E \rightarrow \text{id}$

$E \rightarrow \text{num}$

$E \rightarrow E * E$

$E \rightarrow E / E$

$E \rightarrow E + E$

$E \rightarrow E - E$

$E \rightarrow (E)$

$E \rightarrow E + E . \quad *$

$E \rightarrow E . * E \quad (\text{any})$

- Choose shift (operator precedence)

$E \rightarrow E + E . \quad +$

$E \rightarrow E . + E \quad (\text{any})$

- Choose reduce (left associativity)
- If $a != b != c$ is not allowed, then $!=$ is nonassociative

Yacc: Precedence Directives

- How to specify such disambiguating rules in Yacc?

`%nonassoc EQ NEQ`

`%left PLUS MINUS`

`%left TIMES DIV`

`%right EXP`

- Operators **PLUS** and **MINUS** have the same precedence and are left associative
- Operators **TIMES** and **DIV** have higher precedence than **PLUS** and **MINUS** and are also left associative
- Operator **EXP** is right associative
- Operators **EQ** and **NEQ** have the lowest precedence and are noassociative

Yacc: Precedence Directives

How Yacc resolve conflicts using these precedence directives?

- The precedence declarations (`%left`, etc.) give priorities to the tokens.
 - The priority of shifting a token is given by this token.
 - The priority of a rule is given by the last token occurring on the right-hand side of that rule.
1. If the rule has higher priority, choose reducing
 2. If the token and the rule have equal priority, then:
 1. `%left` favors reducing
 2. `%right` favors shifting
 3. `%nonassoc` yields an error action

Yacc: Precedence Directives

$E \rightarrow E * E .$ +
 $E \rightarrow E . + E$ (any)

```
%nonassoc EQ NEQ  
%left PLUS MINUS  
%left TIMES DIV  
%right EXP
```

- Reduce: *
- Shift: +
- * > +
- => reduce

$E \rightarrow E + E .$ +
 $E \rightarrow E . + E$ (any)

- Reduce: +
- Shift: +
- + = +
- Equal priority, % left => reduce

Yacc: Precedence Directives

- How to parse $-6 * 8$?
 - should be $(-6) * 8$, **not** $-(6 * 8)$
- How to specify this rule?
 - Assign a specific precedence to a rule using the **%prec** directive

```
%{ // declarations of yylex and yyerror
%}
%token INT PLUS MINUS TIMES UMINUS
%start exp
%left PLUS MINUS
%left TIMES
%left UMINUS
%%
exp : INT
    | exp PLUS exp
    | exp MINUS exp
    | exp TIMES exp
    | MINUS exp %prec UMINUS
```

- The directive **%prec UMINUS** gives the rule **exp : MINUS exp** the highest precedence
- The token **UMINUS** is never returned by the Lexer. It is just a placeholder.

Yacc: Syntax v.s. Semantics

- Consider a programming language **with arithmetic expressions** such as $x + y$ and **boolean expressions** such as $a \& (b = c)$

```
%{ //declarations of yylex and yyerror
%}
%token ID ASSIGN PLUS MINUS AND EQUAL
%start stm
%left OR
%left AND
%left PLUS
%%
stm : ID ASSIGN ae
    | ID ASSIGN be
be : be OR be
    | be AND be
    | ae EQUAL ae
    | ID
ae : ae PLUS ae
    | ID
```

- Arithmetic operators bind tighter than the boolean operators
- A boolean expression cannot be added to an arithmetic expression
- There are arithmetic variables and boolean variables

Yacc: Syntax v.s. Semantics

```
%{ //declarations of yylex and yyerror
%}
%token ID ASSIGN PLUS MINUS AND EQUAL
%start stm
%left OR
%left AND
%left PLUS
%%
stm : ID ASSIGN ae
    | ID ASSIGN be
be : be OR be
    | be AND be
    | ae EQUAL ae
    | ID
ae : ae PLUS ae
    | ID
```

state 5: **reduce/reduce conflict**
between rule 6 and
rule 4 on EOF

be : ID .

ae : ID .

conflict

- How to solve this problem?
- Defer this analysis until the “semantic phase” of the compiler.
- e.g., $a + 5 \& b \Rightarrow$ **syntactically correct**

$S \rightarrow id := E$

$E \rightarrow id$

$E \rightarrow E \& E$

$E \rightarrow E = E$

$E \rightarrow E + E$

Error Recovery

- **Motivation:** developers would like to have all the errors in her program reported, not just the first error.
- **Error recovery techniques:**
 - **Local error recovery**
 - **Global error repair**

Error Recovery: Local Error Recovery

- Local error recovery mechanisms work by **adjusting** the parse stack and the input **at the point where the error was detected** in a way that will allow parsing to **resume**.
- One local recovery mechanism used by Yacc:
 - uses a special **error** symbol to control the recovery process

exp → *ID*

exp → *exp* + *exp*

exp → (*exps*)

exps → *exp*

exps → *exps* ; *exp*

exp → (*error*)

exps → *error* ; *exp*

*these are called
synchronizing tokens*

- We can specify that if a syntax error is encountered in the middle of an expression, the parser should skip to the **next semicolon or right parenthesis** and resume parsing.

Error Recovery: Local Error Recovery

$$\begin{array}{l} \textit{exp} \rightarrow (\textit{error}) \\ \textit{exps} \rightarrow \textit{error} ; \textit{exp} \end{array}$$

- What does the parser-generator do with the error symbol?
- *error* is considered a **terminal symbol**, and shift actions are entered in the parsing table for it as if it were an ordinary token.
- How does the parser-generator perform error recover?

Error Recovery: Local Error Recovery

$$exp \rightarrow ID$$
$$exp \rightarrow exp + exp$$
$$exp \rightarrow (exps)$$
$$exprs \rightarrow exp$$
$$exps \rightarrow exps ; exp$$
$$exp \rightarrow (error)$$

exps* $\rightarrow$ *error* ; *exp

input tokens: (ID++)\$

invalid string

[illegible]

Error Recovery: Local Error Recovery

$$exp \rightarrow ID$$
$$exp \rightarrow exp + exp$$
$$exp \rightarrow (exps)$$
$$exprs \rightarrow exp$$
$$exps \rightarrow exps ; exp$$

exp $\rightarrow$ (*error*)

exps* $\rightarrow$ *error* ; *exp

input tokens: (ID++)\$

invalid string

[illegible]

Error Recovery: Local Error Recovery

$exp \rightarrow ID$

$exp \rightarrow exp + exp$

$exp \rightarrow (exps)$

$exps \rightarrow exp$

$exps \rightarrow exps ; exp$

$exp \rightarrow (error)$

$exps \rightarrow error ; exp$

Stack (symbol)	Input	Action
	(ID++)\$	shift
(	ID++)\$	shift
(ID	++)\$	reduce
(exp	++)\$	shift
(exp+	+) \$	error

input tokens: (ID++)\$

invalid string

Error Recovery: Local Error Recovery

$exp \rightarrow ID$

$exp \rightarrow exp + exp$

$exp \rightarrow (exps)$

$exps \rightarrow exp$

$exps \rightarrow exps ; exp$

$exp \rightarrow (error)$

$exps \rightarrow error ; exp$

input tokens: (ID++)\$

invalid string

Stack (symbol)	Input	Action
	(ID++)\$	shift
(	ID++)\$	shift
(ID	++)\$	reduce
(exp	++)\$	shift
(exp+	+) \$	error, pop
(exp	error+) \$	

Error Recovery: Local Error Recovery

exp → *ID*

exp → *exp* + *exp*

exp → (*exps*)

exps → *exp*

exps → *exps* ; *exp*

exp → (*error*)

exps → *error* ; *exp*

Stack (symbol)	Input	Action
	(ID++)\$	shift
(	ID++)\$	shift
(ID	++)\$	reduce
(exp	++)\$	shift
(exp+	+) \$	error, pop
(exp	error+) \$	pop
(	error+) \$	

input tokens: (ID++)\$

invalid string

Error Recovery: Local Error Recovery

$exp \rightarrow ID$

$exp \rightarrow exp + exp$

$exp \rightarrow (exps)$

$exps \rightarrow exp$

$exps \rightarrow exps ; exp$

$exp \rightarrow (error)$

$exps \rightarrow error ; exp$

Stack (symbol)	Input	Action
	(ID++)\$	shift
(	ID++)\$	shift
(ID	++)\$	reduce
(exp	++)\$	shift
(exp+	+) \$	error, pop
(exp	error+) \$	pop
(	error+) \$	shift
(error	+) \$	

input tokens: (ID++)\$

invalid string

Error Recovery: Local Error Recovery

exp → *ID*

exp → *exp* + *exp*

exp → (*exps*)

exps → *exp*

exps → *exps* ; *exp*

exp → (*error*)

exps → *error* ; *exp*

input tokens: (ID++)\$

invalid string

Stack (symbol)	Input	Action
	(ID++)\$	shift
(	ID++)\$	shift
(ID	++)\$	reduce
(exp	++)\$	shift
(exp+	+) \$	error, pop
(exp	error+) \$	pop
(	error+) \$	shift
(error	+) \$	discard input
(error	) \$	

Error Recovery: Local Error Recovery

exp → *ID*

exp → *exp* + *exp*

exp → (*exps*)

exps → *exp*

exps → *exps* ; *exp*

exp → (*error*)

exps → *error* ; *exp*

input tokens: (ID++)\$

invalid string

Stack (symbol)	Input	Action
	(ID++)\$	shift
(	ID++)\$	shift
(ID	++)\$	reduce
(exp	++)\$	shift
(exp+	+) \$	error, pop
(exp	error+) \$	pop
(	error+) \$	shift
(error	+) \$	discard input
(error	) \$	shift
(error)	\$	reduce
exp	\$	

Error Recovery: Local Error Recovery

To summarize, when the LR parser reaches an error state, it takes the following actions:

1. **Pop the stack (if necessary) until a state is reached in which the action for the *error* token is *shift*.**
2. **Shift the *error* token**
3. **Discard input symbols (if necessary) until a state is reached that *has a non-error action* on the current lookahead token.**
4. **Resume normal parsing**

Error Recovery: Local Error Recovery

Caution

- During local error recovery, popping states from the stack can lead to seemingly “impossible” semantic actions, especially if the actions contain side effects.

```
statements: statements exp SEMICOLON
           | statements error SEMICOLON
           | /* empty */
```

```
exp : increment exp decrement
    | ID
```

```
increment: LPAREN {nest=nest+1;}
decrement: RPAREN {nest=nest-1;}
```

1. Pop the stack until we reach the last SEMICOLON or the stack become empty
 2. Shift error
 3. Discard input tokens until we see SEMICOLON
- $nest - 1$ never be executed

- What if a syntax error is found after some left parentheses have been parsed?**
- Solution: have side-effect-free semantic actions (Chap 4.)

Error Recovery: Global Error Repair

typec $\rightarrow$ *type* *type-id* = *ty*

vardec $\rightarrow$ *var* *id* := *exp*

$\rightarrow$ *var* *id* : *type-id* := *exp*

ty $\rightarrow$ *type-id*

$\rightarrow$ { *tyfields* }

$\rightarrow$ *array of* *type-id*

let type a := intArray [10] of 0 in ...

- *intArray* [10] of 0 returns a new array of *intArray* with 10 elements, and each element is initialized as 0.
- How does local error recovery techniques recover this error?
- A local technique will discover a syntax error with := as lookahead symbol:
 - delete the phrase from *type* to 0
 - or, replace the := by =, and will encounter another syntax error at the [token

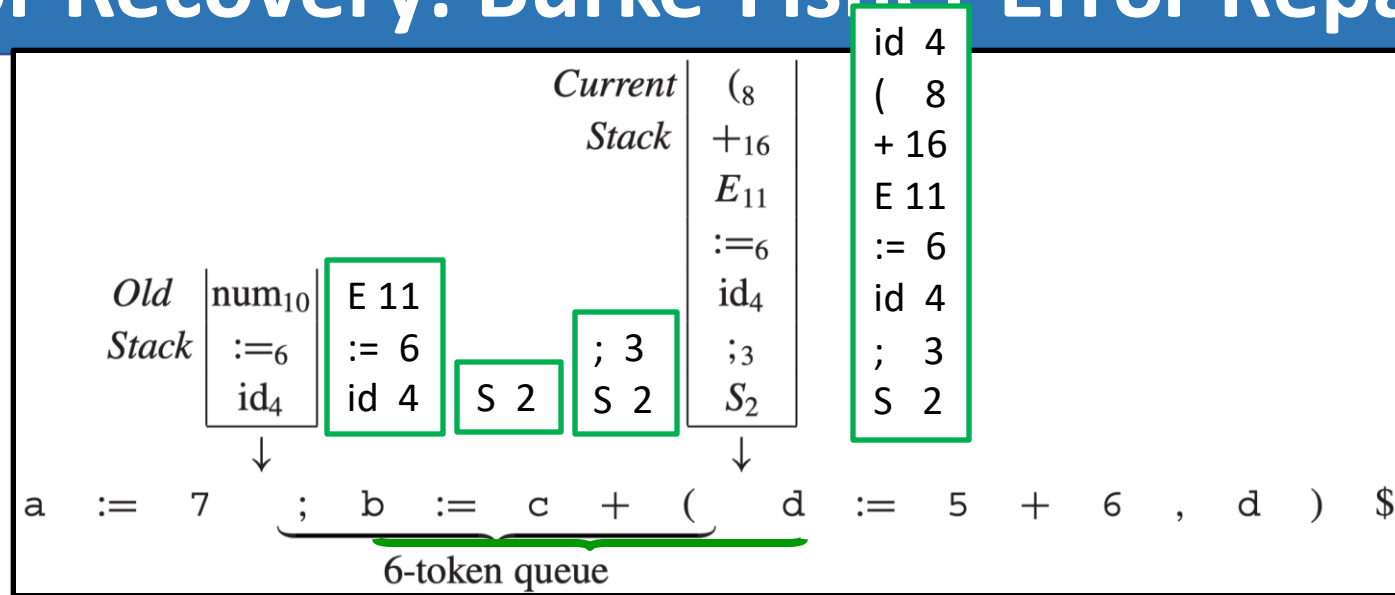
Error Recovery: Global Error Repair

- We need some techniques that can find the root cause, i.e., `type` should be `var`, even if the root cause is not at a position where the parser would **first report the error**.
- *Global error repair* finds the smallest set of insertions and deletions that would turn the source string into a syntactically correct string, *even if the insertions and deletions are not at a point where an LL or LR parser would first report an error*.

Error Recovery: Burke-Fisher Error Repair

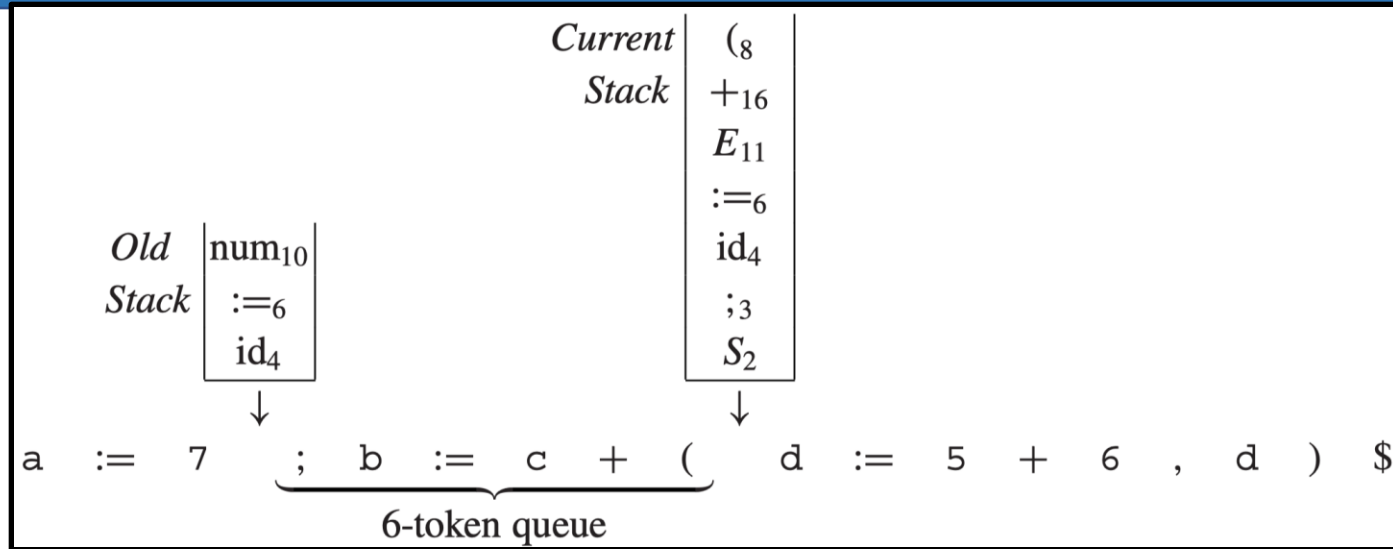
- Burke-Fisher error repair: tries every possible single-token insertion, deletion, or replacement at every point that occurs no earlier than K tokens before the point where the parser reported the error:
 - limited but useful
- How to choose the best error repair?
 - The correction that allows the parser to parse furthest past the original reported error.
 - Generally, if a repair carries the parser $R = 4$ tokens beyond where it originally got stuck, this is “good enough.”
- The parsing engine must be able to back up K tokens and reparse.
- How to implement this technique?

Error Recovery: Burke-Fisher Error Repair



- Maintain **two parse stacks**: the **current stack** and the **old stack**.
- Keep **a queue of K tokens**
- As each new token is shifted:
 - It is pushed on the **current stack** and put onto **the tail of the queue**
 - The **head of the queue** is removed and **shifted onto the old stack**
- With **each shift** onto the old or current stack, the **appropriate reduce actions** are also performed.

Error Recovery: Burke-Fisher Error Repair



- Suppose a syntax error is detected at the *current* token:
 - For **each possible insertion, deletion, or substitution** of a token at any position of the queue, the Burke-Fisher error repairer makes that change to within **(a copy of) the queue**, then **attempts to reparse from the old stack**.
 - Generally, if **three or four** tokens past the *current* token can be parsed, this is considered a completely successful repair.

The end of Chapter 3